Assignment 1: Technique Practice and Prompt Diagnosis

10% · Due: End of Week 8 — Nov 1, 23:59

> **Due:** end of Week 8 — Sunday, November 1, by the end of day (23:59). **Weight:** 10%.

The Engineer's Role in Agentic Engineering

The engineer's job is to be the decision-maker at every level: specifying intent clearly enough for the agent to execute, evaluating the agent's trajectory (not just its individual outputs) to detect when things are going wrong, and knowing when to take back control. This framing applies whether you are writing a single prompt, building a RAG pipeline, or directing a multi-step agent. The tool changes; the engineer's responsibility does not.

This is the first of three individual assignments across the semester. They form a progressive arc: use techniques deliberately (A1), design and verify a system (A2), demonstrate growth in engineering judgment with evidence (A3).

Note on AI Assistance

In this assignment, you are required to use AI to work on this assignment — leveraging the prompt techniques that you learned from the lecture to use AI tools to debug, build, and document in a simulated software engineering task.

In addition to the prompt engineering practice, you may also use AI tools to help with this assignment. If you do, document this in the assignment as the **AI usage log**. The reflection and diagnosis requirements are where the learning is — these require you to actually run prompts and read outputs carefully. Generating reflections through AI without doing the underlying work will be evident in the specificity of your analysis.

Overview

There are two parts in this assignment:

- **Part 1:** use prompting techniques deliberately and articulate what each adds.
- **Part 2:** diagnose broken prompts and fix them with documented reasoning.

Together, the two parts help you examine: can you specify intent precisely, and recognise when a specification is failing?

Score Distribution (100 points)

QUESTION	POINTS
Part 1: Deliberate Technique Practice	**50**
Total	**100**

QUESTION	POINTS
Task 1: Debug `get_overdue_tasks` (Chain-of-Thought: 10 points, Step-back: 10 points)	20
Task 2: Build `sort_tasks_by_priority` (Zero-shot: 10 points, Self-correction: 10 points)	20
Task 3: Document `calculate_completion_rate` (Few-shot: 10 points)	10
Part 2: Prompt Diagnosis and Rewrite	**50**
Prompt 1	15
Prompt 2	15
Prompt 3	15
Integrity of evidence across Part 2	5
Total	**100**

How each question is scored is described under *Grading Criteria* below.

Submission Format (Parts 1 and 2)

Submit your work for **both parts** as a single Markdown file using the sample submission template, which provides the headings for Part 1, Part 2 and the AI usage log. Keep the headings and replace the grey instructions with your own work.

> Your final submission file must be named `assignment_1_studentid_name.md` .

> **Assignment 1 materials** (including the sample submission template): posted on E3.

Part 1 — Deliberate Technique Practice (50%)

All three tasks below share one small module, `task_manager.py` . The assignment is a small project rather than three disconnected exercises. Five techniques are spread across three task types — each mirroring a different kind of real programming work — rather than one technique repeated five times on the same task.

Provided module (`task_manager.py`):

```
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.

    Args:
        tasks (list): The list of existing task dictionaries.
```

```
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority, "completed": False})
    return tasks

def mark_complete(tasks, title):
    """Mark a task as completed by title.

    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
            task["completed"] = True
            return True
    return False

def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

Task 1 — Debug `get_overdue_tasks` *(techniques: Chain-of-Thought, Step-back)*

`get_overdue_tasks` crashes with `TypeError: '<' not supported between instances of 'NoneType' and 'datetime.date'` the moment it's given any task created without a due date — which `add_task` explicitly allows by default. This is a verified, deterministic bug, not a hypothetical one.

Your task: write the Chain-of-Thought prompt and Step-back prompt to help debug and produce code that will resolve the issue.

Task 2 — Build `sort_tasks_by_priority` *(techniques: Zero-shot, Self-correction)*

Spec (verified against a reference solution): return a new list sorted so higher-priority tasks come first, earlier due dates come first within the same priority, and tasks with no due date sort last within their priority tier.

Your task: write the Zero-shot prompt and Self-correction prompts to produce code that will successfully run the case.

Task 3 — Document `calculate_completion_rate` *(technique: Few-shot)*

Your task: write the Few-shot prompt to produce documentation (the docstring) for the function.

What to Submit for Part 1

For each of the five technique applications above: the prompt verbatim, the full model response, and two to three sentences on what the technique added or failed to add for that specific task. Please see the sample submission template for the detailed format.

Part 2 — Prompt Diagnosis and Rewrite (50%)

Imagine you are working with others for a software project within a team. Below are three prompts written by a teammate of yours. Each one leaves something out, so the code the model returns does not do what the author wanted. **You decide what went wrong.**

How to Run Each Prompt

1. **Original run.** Open a fresh conversation. Paste the full `task_manager.py`, then the prompt. For this first run, do **not** paste `CONVENTIONS.md` or `utils.py` or the author's intent: the teammate wrote the prompt as if the model already had them, and whether it did is part of what you are diagnosing. Read `CONVENTIONS.md` and `utils.py` only after this first run.
2. **Check it.** Copy the code the model gives you into `my_solutions.py` (using the function name in the prompt) and run `python check_my_work.py`. Save the code and the checker output. FAIL lines here are expected: they are your evidence of what went wrong, not a mistake on your part.
3. **Diagnose and rewrite.** Work out *why* the model produced what it did, then write an improved prompt. Your rewrite may use anything in `CONVENTIONS.md`, `utils.py` or the author's intent below.
4. **Rewrite run.** Run your rewrite in a fresh conversation, replace the code in `my_solutions.py`, run the checker again, and save the output.

For each prompt, submit:

- **Failure mode:** name it precisely. “It is vague” is not a failure mode; the specific thing that went wrong is.
- **Mechanism:** explain *why* it happened, using course vocabulary from Weeks 1–7 and pointing to something in your own output and checker results.
- **Rewritten prompt:** your improved version.
- **Evidence:** for both the original and the rewritten prompt, the full code the model gave you and the checker output.
- **Personal connection:** one sentence. Have you seen a similar failure in your own AI use?

> **If the model asks you a question instead of writing code,** that is a real result. Say so, then reply “Please proceed using your best assumptions” and use the code it gives you. Your diagnosis should say what it asked about and why the prompt left that gap.

> **If the original code passes every check,** that is also a real result. Say so, show the output, and explain what the model assumed and why those assumptions happened to be right. Run the original prompt twice more in fresh

> chats and report whether the model made the same assumptions each time. Then rewrite the prompt so it no longer depends on the model guessing.

Prompt 1 (15 points)

> *"Write a function `add_subtask(tasks, parent_title, subtask_title)` that adds a subtask to an existing task."*

What the author wanted: the team has a file called `CONVENTIONS.md`, and every function must follow it. The author wrote the prompt as if everyone had read it.

Prompt 2 (15 points)

> *"Write a function `format_task_line(task, today)` that returns a one-line display string for a task, showing its title, priority and due date. Use our existing date-formatting helper in `utils.py` for the due date."*

What the author wanted: the due-date wording to come from the team's own helper, so that every screen words due dates the same way.

Prompt 3 (15 points)

> *"Write a function `get_urgent_tasks(tasks, today)` that returns the urgent tasks, most urgent first."*

What the author wanted: a task is **urgent** if it is not completed and at least one of these is true: it is overdue; it is due within the next 3 days, counting today; or its priority is `"high"` (even with no due date). "Most urgent first" meant earliest due date first, with undated high-priority tasks last. The function must not modify the list it is given.

Grading Criteria (100 points)

Each criterion below shows what earns full credit, partial credit and no credit. The checker (`check_my_work.py`) helps you see whether your code works, but it does not award points. Points come from your prompts, the model's responses, your reflections and your diagnoses.

Part 1 (50 points): five technique applications, 10 points each

Applies to each of: Chain-of-Thought, Step-back, Zero-shot, Self-correction and Few-shot.

CRITERION	PTS	FULL CREDIT	PARTIAL CREDIT	NO CREDIT
Technique used as specified	4	The prompt you submit contains the technique's defining feature (below). Where two techniques share a task, the two prompts are meaningfully different from each other.	The technique is recognisable but diluted, or the two prompts on the same task are near-duplicates (2).	Wrong technique, or the prompt is not shown (0).

CRITERION	PTS	FULL CREDIT	PARTIAL CREDIT	NO CREDIT
Full response included	2	The full, unedited model response is shown. For self-correction, all three turns (generate, critique, revise).	The response is truncated or paraphrased (1).	The response is missing (0).
Reflection	4	2–3 sentences, specific to this task and this output; says what the technique added *or failed to add*; every claim matches the response you show.	Plausible but generic, could describe any task (2). Or one claim contradicts the response you show (2).	Generic filler, or contradicts the response you show throughout (0).

What each technique must contain

- **Chain-of-Thought:** the prompt asks the model to reason or trace step by step before answering.
- **Step-back:** the prompt asks the model for the general principle first, then applies it to the specific case.
- **Zero-shot:** no examples and no reasoning scaffold, just a plain, complete specification.
- **Self-correction:** three turns are shown: generate, a *specific* critique (not just "is this good?"), and a revision.
- **Few-shot:** both provided docstrings (`add_task` , `mark_complete`) are used as examples.

A reflection that says a technique added nothing, when that is true, earns full credit. Generic reflections that could describe any task score lower.

Part 2 (50 points): three prompts, 15 points each, plus 5

CRITERION	PTS PER PROMPT	FULL CREDIT	PARTIAL CREDIT	NO CREDIT
Failure mode	3	Named precisely (not "vague" or "bad prompt") and consistent with your own transcript and checker output. If the model asked a question, or your original code passed every check, you say so plainly and follow the instructions in Part 2.	Correct category but imprecise name (2). Or an unusual outcome reported without the explanation Part 2 asks for (1).	Contradicts your checker output, or only says "it was bad" (0).
Mechanism	4	A causal link from the specific gap in the prompt to what the model produced, using course vocabulary from Weeks 1–7 accurately and pointing to something in your own output.	Correct vocabulary but no causal link, or only restates the symptom (2). Or one term used loosely (3).	Vocabulary absent or wrong (0–1).
Rewritten prompt	3	Fixes the cause you diagnosed (supplies the missing information, or states it explicitly), not just rewording or "be careful".	Partly addresses the cause (1–2).	Reworded only (0).
Evidence	4	The full code and the checker output from **both** the original and	One output missing or truncated (2). The	No outputs, or outputs that

CRITERION	PTS PER PROMPT	FULL CREDIT	PARTIAL CREDIT	NO CREDIT
		the rewrite, from real runs and consistent with each other; the rewrite's checker output is better in the way your diagnosis predicts.	difference is real but not connected to your diagnosis (2–3).	contradict your claim (0).
Personal connection	1	One sentence naming a concrete, plausible experience of a similar failure.		Missing (0).

Integrity of evidence (5 points, across all of Part 2):

FULL CREDIT (5)	PARTIAL CREDIT (2–4)	NO CREDIT (0)
Your diagnoses match your transcripts and checker output, the outputs look like genuine model output, and where a prompt did not fail for you, you say so.	One diagnosis or output is inconsistent with its transcript, or part of the evidence is missing.	Fabricated outputs, or diagnoses that describe failures your outputs do not show.

← Back to Assignments